

```
/*
 * AIRouter V1.0.0
 * 著作权人: 陆豪
 */
import { rcp } from '@kit.RemoteCommunicationKit';

const DEFAULT_RESUME_CWD: string = '~';
const OPENCODE_RESUMES_PATH: string = '/api/opencode/resumes';
const FILES_PATH: string = '/api/files';

export interface LoginRequest {
  baseUrl: string;
  registrationCode: string;
  deviceName: string;
  deviceId: string;
  publicKey: string;
}

export interface RegisterRequest {
  baseUrl: string;
  deviceId: string;
  publicKey: string;
  registrationCode: string;
  deviceName: string;
}

export interface RegisterResponse {
  activationToken: string;
  challenge: string;
  registrationCode?: string;
}

export interface ActivateRequest {
  baseUrl: string;
  deviceId: string;
  activationToken: string;
  signedChallenge: string;
}

export interface ChallengeRequest {
  baseUrl: string;
  deviceId: string;
}

export interface ChallengeResponse {
  nonce: string;
  expiresAt: string;
}

export interface VerifyRequest {
```

```
    baseUrl: string;
    deviceId: string;
    nonce: string;
    signature: string;
}

export interface RefreshRequest {
    baseUrl: string;
    refreshToken: string;
}

export interface RefreshResponse {
    accessToken: string;
}

export interface LogoutRequest {
    baseUrl: string;
    refreshToken: string;
}

export interface LoginResponse {
    accessToken: string;
    refreshToken: string;
    sessionName: string;
    expiresIn: number;
    registrationCode?: string;
}

export interface ListResumesRequest {
    baseUrl: string;
    accessToken: string;
}

export interface StartResumeRequest {
    baseUrl: string;
    accessToken: string;
    cwd?: string;
}

export interface ResumeThreadRequest {
    baseUrl: string;
    accessToken: string;
    threadId: string;
}

export interface SendResumeMessageRequest {
    baseUrl: string;
    accessToken: string;
    threadId: string;
    message: string;
}
```

```
}

export interface ResumeMutationRequest {
  baseUrl: string;
  accessToken: string;
  threadId: string;
}

export interface RenameResumeRequest extends ResumeMutationRequest {
  name: string;
}

export interface ListFilesRequest {
  baseUrl: string;
  accessToken: string;
}

export interface ServerFile {
  name: string;
  size: number;
  modifiedAt: string;
}

export interface DownloadFileRequest {
  baseUrl: string;
  accessToken: string;
  fileName: string;
}

export interface DownloadedFile {
  name: string;
  bytes: ArrayBuffer;
}

export interface ResumeItem {
  id: string;
  title: string;
  subtitle: string;
  status: string;
  updatedAt: string;
}

export interface ResumeSession {
  threadId: string;
  title: string;
  cwd: string;
  status: string;
  turns: ResumeTurn[];
}
```

```
export interface ResumeTurn {
  id: string;
  status: string;
  startedAt: string;
  completedAt: string;
  items: ResumeContentItem[];
}

export interface ResumeContentItem {
  id: string;
  role: string;
  kind: string;
  text: string;
  status: string;
}

export interface GatewayClient {
  register(request: RegisterRequest): Promise<RegisterResponse>;
  activate(request: ActivateRequest): Promise<LoginResponse>;
  challenge(request: ChallengeRequest): Promise<ChallengeResponse>;
  verify(request: VerifyRequest): Promise<LoginResponse>;
  refresh(request: RefreshRequest): Promise<RefreshResponse>;
  logout(request: LogoutRequest): Promise<void>;
  checkHealth(baseUrl: string): Promise<boolean>;
  listResumes(request: ListResumesRequest): Promise<ResumeItem[]>;
  startResume(request: StartResumeRequest): Promise<ResumeSession>;
  resumeThread(request: ResumeThreadRequest): Promise<ResumeSession>;
  sendResumeMessage(request: SendResumeMessageRequest): Promise<ResumeSession>;
  renameResume(request: RenameResumeRequest): Promise<void>;
  deleteResume(request: ResumeMutationRequest): Promise<void>;
  listFiles(request: ListFilesRequest): Promise<ServerFile[]>;
  downloadFile(request: DownloadFileRequest): Promise<DownloadedFile>;
}

export interface RequestHeaders {
  contentType: string;
  authorization: string;
}

export interface RequestOptions {
  method: string;
  headers: RequestHeaders;
  body?: string;
}

export interface GatewayHttpResponse {
  statusCode: number;
  result: string;
}
```

```
export interface GatewayBinaryResponse {
  statusCode: number;
  body: ArrayBuffer;
}

export interface GatewayTransport {
  request(url: string, options: RequestOptions): Promise<GatewayHttpResponse>;
  requestBinary(url: string, options: RequestOptions): Promise<GatewayBinaryResponse>;
}

export class GatewayHttpError extends Error {
  public statusCode: number;

  constructor(statusCode: number) {
    super(`Gateway request failed: ${statusCode}`);
    this.statusCode = statusCode;
  }
}

export class GatewayNetworkError extends Error {
  public detail: string;

  constructor(detail: string = '') {
    super('Gateway network request failed');
    this.detail = detail;
  }
}

export function isGatewayAuthenticationError(error: Object): boolean {
  if (!(error instanceof GatewayHttpError)) {
    return false;
  }

  return error.statusCode === 401 || error.statusCode === 403;
}

export function isGatewayNetworkError(error: Object): boolean {
  return error instanceof GatewayNetworkError;
}

interface LoginResponseBody {
  accessToken: string;
  refreshToken: string;
  sessionName: string;
  expiresIn: number;
  registrationCode?: string;
}

interface RegisterResponseBody {
  activationToken: string;
}
```

```
    challenge: string;
    registrationCode?: string;
}

interface ChallengeResponseBody {
    nonce: string;
    expiresAt: string;
}

interface RefreshResponseBody {
    accessToken: string;
}

interface HealthResponseBody {
    status: string;
}

interface ListResumesResponseBody {
    items: ResumeItem[];
}

interface ListFilesResponseBody {
    items: ServerFile[];
}

interface ThreadStatus {
    type?: string;
}

interface ResumeThreadBodyThread {
    id?: string;
    name?: string;
    preview?: string;
    cwd?: string;
    status?: string | ThreadStatus;
    turns?: ResumeTurn[];
}

interface ResumeThreadResponseBody {
    threadId?: string;
    title?: string;
    cwd?: string;
    status?: string | ThreadStatus;
    turns?: ResumeTurn[];
    thread?: ResumeThreadBodyThread;
}

class SystemGatewayTransport implements GatewayTransport {
    private session: rcp.Session;
    private caContent: string;
```

```
constructor(caContent: string = '') {
  this.caContent = caContent;
  const timeoutCfg: rcp.Timeout = {
    connectMs: 30000,
    transferMs: 30000
  };
  if (caContent.length > 0) {
    console.info(`[SystemGatewayTransport] Creating session with CA cert, length=${caContent.length}`);
    const caCert: rcp.CertificateAuthority = { content: this.caContent };
    const sessionCfg: rcp.SessionConfiguration = {
      requestConfiguration: {
        transfer: {
          timeout: timeoutCfg
        },
        security: { remoteValidation: caCert }
      }
    };
    this.session = rcp.createSession(sessionCfg);
  } else {
    console.info('[SystemGatewayTransport] Creating session WITHOUT CA cert (default trust)');
    const sessionCfg: rcp.SessionConfiguration = {
      requestConfiguration: {
        transfer: {
          timeout: timeoutCfg
        }
      }
    };
    this.session = rcp.createSession(sessionCfg);
  }
}

async request(url: string, options: RequestOptions): Promise<GatewayHttpResponse> {
  const req = new rcp.Request(url, options.method);

  const headers: Record<string, string> = {};
  if (options.headers.contentType.length > 0) {
    headers['content-type'] = options.headers.contentType;
  }
  if (options.headers.authorization.length > 0) {
    headers['authorization'] = options.headers.authorization;
  }
  req.headers = headers;

  if (options.body !== undefined && options.body.length > 0) {
    req.content = options.body;
  }

  try {
    const response = await this.session.fetch(req);
  }
}
```

```

        const result: string = JSON.stringify(response.toJSON());

        return {
            statusCode: response.statusCode,
            result: result
        };
    } catch (error) {
        const detail: string = typeof error === 'object' && error !== null ? JSON.stringify(error) : String(error);
        console.error(`[GatewayTransport] request failed: url=${url}, method=${options.method}, error=${detail}`);
        throw new GatewayNetworkError(detail);
    }
}

async requestBinary(url: string, options: RequestOptions): Promise<GatewayBinaryResponse> {
    const req = new rcp.Request(url, options.method);

    const headers: Record<string, string> = {};
    if (options.headers.contentType.length > 0) {
        headers['content-type'] = options.headers.contentType;
    }
    if (options.headers.authorization.length > 0) {
        headers['authorization'] = options.headers.authorization;
    }
    req.headers = headers;

    try {
        const response = await this.session.fetch(req);
        return {
            statusCode: response.statusCode,
            body: response.body !== undefined ? response.body : new ArrayBuffer(0)
        };
    } catch (error) {
        const detail: string = typeof error === 'object' && error !== null ? JSON.stringify(error) : String(error);
        console.error(`[GatewayTransport] requestBinary failed: url=${url}, method=${options.method}, error=${detail}`);
        throw new GatewayNetworkError(detail);
    }
}

export class HttpGatewayClient implements GatewayClient {
    private transport: GatewayTransport;

    constructor(transport: GatewayTransport = new SystemGatewayTransport()) {
        this.transport = transport;
    }

    static createWithCert(certContent: string): HttpGatewayClient {
        return new HttpGatewayClient(new SystemGatewayTransport(certContent));
    }
}

```

```
async register(request: RegisterRequest): Promise<RegisterResponse> {
  const response = await this.transport.request(joinUrl(request.baseUrl, '/api/auth/register'), {
    method: 'POST',
    headers: {
      contentType: 'application/json',
      authorization: ''
    },
    body: JSON.stringify({
      deviceId: request.deviceId,
      publicKey: request.publicKey,
      registrationCode: request.registrationCode,
      deviceName: request.deviceName
    })
  });
  ensureSuccess(response);

  const body = JSON.parse(response.result) as RegisterResponseBody;
  return {
    activationToken: body.activationToken,
    challenge: body.challenge,
    registrationCode: body.registrationCode
  };
}

async activate(request: ActivateRequest): Promise<LoginResponse> {
  const response = await this.transport.request(joinUrl(request.baseUrl, '/api/auth/activate'), {
    method: 'POST',
    headers: {
      contentType: 'application/json',
      authorization: ''
    },
    body: JSON.stringify({
      deviceId: request.deviceId,
      activationToken: request.activationToken,
      signedChallenge: request.signedChallenge
    })
  });
  ensureSuccess(response);

  const body = JSON.parse(response.result) as LoginResponseBody;
  return {
    accessToken: body.accessToken,
    refreshToken: body.refreshToken,
    sessionName: 'opencode-main',
    expiresIn: 900,
    registrationCode: body.registrationCode
  };
}

async challenge(request: ChallengeRequest): Promise<ChallengeResponse> {
```

```
const response = await this.transport.request(joinUrl(request.baseUrl, '/api/auth/challenge'), {
  method: 'POST',
  headers: {
    contentType: 'application/json',
    authorization: ''
  },
  body: JSON.stringify({
    deviceId: request.deviceId
  })
});
ensureSuccess(response);

const body = JSON.parse(response.result) as ChallengeResponseBody;
return {
  nonce: body.nonce,
  expiresAt: body.expiresAt
};
}

async verify(request: VerifyRequest): Promise<LoginResponse> {
  const response = await this.transport.request(joinUrl(request.baseUrl, '/api/auth/verify'), {
    method: 'POST',
    headers: {
      contentType: 'application/json',
      authorization: ''
    },
    body: JSON.stringify({
      deviceId: request.deviceId,
      nonce: request.nonce,
      signature: request.signature
    })
  });
  ensureSuccess(response);

  const body = JSON.parse(response.result) as LoginResponseBody;
  return {
    accessToken: body.accessToken,
    refreshToken: body.refreshToken,
    sessionName: 'opencode-main',
    expiresIn: 900
  };
}

async refresh(request: RefreshRequest): Promise<RefreshResponse> {
  const response = await this.transport.request(joinUrl(request.baseUrl, '/api/auth/refresh'), {
    method: 'POST',
    headers: {
      contentType: 'application/json',
      authorization: ''
    },
  },
```

```
        body: JSON.stringify({
            refreshToken: request.refreshToken
        })
    });
    ensureSuccess(response);

    const body = JSON.parse(response.result) as RefreshResponseBody;
    return {
        accessToken: body.accessToken
    };
}

async logout(request: LogoutRequest): Promise<void> {
    try {
        await this.transport.request(joinUrl(request.baseUrl, '/api/auth/logout'), {
            method: 'POST',
            headers: {
                contentType: 'application/json',
                authorization: ''
            },
            body: JSON.stringify({
                refreshToken: request.refreshToken
            })
        });
    } catch {
        // Best-effort
    }
}

async checkHealth(baseUrl: string): Promise<boolean> {
    const response = await this.transport.request(joinUrl(baseUrl, '/health'), {
        method: 'GET',
        headers: {
            contentType: '',
            authorization: ''
        }
    });
    if (response.statusCode !== 200) {
        return false;
    }
    try {
        const body = JSON.parse(response.result) as HealthResponseBody;
        return body.status === 'ok';
    } catch {
        return false;
    }
}

async listResumes(request: ListResumesRequest): Promise<ResumeItem[]> {
    const url = joinUrl(request.baseUrl, OPENCODE_RESUMES_PATH);
```

```
console.info(`[listResumes] 请求 URL: ${url}`);
const response = await this.transport.request(url, {
  method: 'GET',
  headers: {
    contentType: '',
    authorization: `Bearer ${request.accessToken}`
  }
});
ensureSuccess(response);

const body = JSON.parse(response.result) as ListResumesResponseBody;
return body.items;
}

async startResume(request: StartResumeRequest): Promise<ResumeSession> {
  const response = await this.transport.request(joinUrl(request.baseUrl, OPENCODE_RESUMES_PATH), {
    method: 'POST',
    headers: {
      contentType: 'application/json',
      authorization: `Bearer ${request.accessToken}`
    },
    body: JSON.stringify({
      cwd: normalizeString(request.cwd)
    })
  });
  ensureSuccess(response);

  return toResumeSession(JSON.parse(response.result) as ResumeThreadResponseBody, '');
}

async resumeThread(request: ResumeThreadRequest): Promise<ResumeSession> {
  const response = await this.transport.request(
    joinUrl(request.baseUrl, `${OPENCODE_RESUMES_PATH}/${encodeURIComponent(request.threadId)}/resume`),
    {
      method: 'POST',
      headers: {
        contentType: 'application/json',
        authorization: `Bearer ${request.accessToken}`
      },
      body: JSON.stringify({
        threadId: request.threadId
      })
    }
  );
  ensureSuccess(response);

  return toResumeSession(JSON.parse(response.result) as ResumeThreadResponseBody, request.threadId);
}

async sendResumeMessage(request: SendResumeMessageRequest): Promise<ResumeSession> {
```

```
const response = await this.transport.request(
  joinUrl(request.baseUrl, `${OPENCODE_RESUMES_PATH}/${encodeURIComponent(request.threadId)}/messages`),
  {
    method: 'POST',
    headers: {
      contentType: 'application/json',
      authorization: `Bearer ${request.accessToken}`
    },
    body: JSON.stringify({
      message: request.message
    })
  }
);
ensureSuccess(response);

return toResumeSession(JSON.parse(response.result) as ResumeThreadResponseBody, request.threadId);
}

async renameResume(request: RenameResumeRequest): Promise<void> {
  const response = await this.transport.request(
    joinUrl(request.baseUrl, `${OPENCODE_RESUMES_PATH}/${encodeURIComponent(request.threadId)}/name`),
    {
      method: 'POST',
      headers: {
        contentType: 'application/json',
        authorization: `Bearer ${request.accessToken}`
      },
      body: JSON.stringify({
        name: request.name
      })
    }
  );
  ensureSuccess(response);
}

async deleteResume(request: ResumeMutationRequest): Promise<void> {
  const response = await this.transport.request(
    joinUrl(request.baseUrl, `${OPENCODE_RESUMES_PATH}/${encodeURIComponent(request.threadId)}`),
    {
      method: 'DELETE',
      headers: {
        contentType: '',
        authorization: `Bearer ${request.accessToken}`
      }
    }
  );
  ensureSuccess(response);
}

async listFiles(request: ListFilesRequest): Promise<ServerFile[]> {
```

```
const url = joinUrl(request.baseUrl, FILES_PATH);
console.info(`[listFiles] 请求 URL: ${url}`);
const response = await this.transport.request(url, {
  method: 'GET',
  headers: {
    contentType: '',
    authorization: `Bearer ${request.accessToken}`
  }
});
ensureSuccess(response);

const body = JSON.parse(response.result) as ListFilesResponseBody;
return body.items;
}

async downloadFile(request: DownloadFileRequest): Promise<DownloadedFile> {
  const url = joinUrl(request.baseUrl, `${FILES_PATH}/${encodeURIComponent(request.fileName)}/download`);
  console.info(`[downloadFile] 请求 URL: ${url}`);
  const response = await this.transport.requestBinary(url, {
    method: 'GET',
    headers: {
      contentType: '',
      authorization: `Bearer ${request.accessToken}`
    }
  });
  ensureSuccessCode(response.statusCode);

  return {
    name: request.fileName,
    bytes: response.body
  };
}

export class MockGatewayClient implements GatewayClient {
  async checkHealth(baseUrl: string): Promise<boolean> {
    return true;
  }

  async register(request: RegisterRequest): Promise<RegisterResponse> {
    return {
      activationToken: 'act-mock-001',
      challenge: 'mock-challenge'
    };
  }

  async activate(request: ActivateRequest): Promise<LoginResponse> {
    return {
      accessToken: 'router_access_mock',
      refreshToken: 'router_refresh_mock',
    };
  }
}
```

```
        sessionName: 'opencode-main',
        expiresIn: 900
    });
}

async challenge(request: ChallengeRequest): Promise<ChallengeResponse> {
    return {
        nonce: 'mock-nonce',
        expiresAt: new Date(Date.now() + 120000).toISOString()
    };
}

async verify(request: VerifyRequest): Promise<LoginResponse> {
    return {
        accessToken: 'router_access_mock',
        refreshToken: 'router_refresh_mock',
        sessionName: 'opencode-main',
        expiresIn: 900
    };
}

async refresh(request: RefreshRequest): Promise<RefreshResponse> {
    return {
        accessToken: 'router_access_mock_refreshed'
    };
}

async logout(request: LogoutRequest): Promise<void> {
}

async listResumes(request: ListResumesRequest): Promise<ResumeItem[]> {
    return [
        {
            id: 'thread-login-fix',
            title: 'Fix router login',
            subtitle: '/srv/projects/codex-router',
            status: 'completed',
            updatedAt: '2026-07-02T10:54:44.000Z'
        },
        {
            id: 'thread-backend',
            title: 'Bridge OpenCode app-server',
            subtitle: '/srv/projects/codex-router',
            status: 'running',
            updatedAt: '2026-07-02T11:20:00.000Z'
        }
    ];
}

async startResume(request: StartResumeRequest): Promise<ResumeSession> {
```

```

    return {
      threadId: 'thread-new',
      title: 'New OpenCode Resume',
      cwd: normalizeString(request.cwd) || DEFAULT_RESUME_CWD,
      status: 'running',
      turns: []
    };
  }

  async resumeThread(request: ResumeThreadRequest): Promise<ResumeSession> {
    return {
      threadId: request.threadId,
      title: request.threadId === 'thread-login-fix' ? 'Fix router login' : 'Bridge OpenCode app-server',
      cwd: '/srv/projects/codex-router',
      status: 'running',
      turns: [
        {
          id: 'turn-mock',
          status: 'completed',
          startedAt: '2026-07-02T10:54:44.000Z',
          completedAt: '2026-07-02T10:55:00.000Z',
          items: [
            {
              id: 'turn-mock:item-1',
              role: 'user',
              kind: 'message',
              text: 'Open saved OpenCode session',
              status: ''
            },
            {
              id: 'turn-mock:item-2',
              role: 'assistant',
              kind: 'message',
              text: 'Loaded session content from OpenCode.',
              status: ''
            }
          ]
        }
      ]
    };
  }

  async sendResumeMessage(request: SendResumeMessageRequest): Promise<ResumeSession> {
    const session = await this.resumeThread({
      baseUrl: request.baseUrl,
      accessToken: request.accessToken,
      threadId: request.threadId
    });
    return {
      threadId: session.threadId,

```

```
title: session.title,
cwd: session.cwd,
status: 'running',
turns: session.turns.concat([
  {
    id: 'turn-mock-reply',
    status: 'completed',
    startedAt: '2026-07-02T11:00:00.000Z',
    completedAt: '2026-07-02T11:01:00.000Z',
    items: [
      {
        id: 'turn-mock-reply:item-1',
        role: 'user',
        kind: 'message',
        text: request.message,
        status: ''
      },
      {
        id: 'turn-mock-reply:item-2',
        role: 'assistant',
        kind: 'message',
        text: 'Mock reply from OpenCode.',
        status: ''
      }
    ]
  }
])
});
}

async renameResume(_request: RenameResumeRequest): Promise<void> {
}

async deleteResume(_request: ResumeMutationRequest): Promise<void> {
}

async listFiles(_request: ListFilesRequest): Promise<ServerFile[]> {
  return [
    {
      name: '红楼梦第一节课.pptx',
      size: 39610,
      modifiedAt: '2026-09-02T08:43:38.000Z'
    },
    {
      name: '调研报告.docx',
      size: 25410,
      modifiedAt: '2026-09-01T03:20:00.000Z'
    }
  ];
}
```

```
async downloadFile(_request: DownloadFileRequest): Promise<DownloadedFile> {
  return {
    name: _request.fileName,
    bytes: new ArrayBuffer(0)
  };
}

function toResumeSession(body: ResumeThreadResponseBody, fallbackThreadId: string): ResumeSession {
  const thread = body.thread;
  const status = body.status !== undefined ? body.status : thread?.status;
  const title = normalizeString(body.title) || normalizeString(thread?.name) || normalizeString(thread?.preview)
  ||
  fallbackThreadId;

  return {
    threadId: normalizeString(body.threadId) || normalizeString(thread?.id) || fallbackThreadId,
    title,
    cwd: normalizeString(body.cwd) || normalizeString(thread?.cwd),
    status: normalizeStatus(status),
    turns: normalizeTurns(body.turns !== undefined ? body.turns : thread?.turns)
  };
}

function normalizeTurns(value: ResumeTurn[] | undefined): ResumeTurn[] {
  if (!Array.isArray(value)) {
    return [];
  }

  return value.map((turn: ResumeTurn, turnIndex: number) => {
    const turnId = normalizeString(turn.id) || `turn-${turnIndex + 1}`;
    const normalized: ResumeTurn = {
      id: turnId,
      status: normalizeString(turn.status) || 'unknown',
      startedAt: normalizeString(turn.startedAt),
      completedAt: normalizeString(turn.completedAt),
      items: normalizeItems(turn.items, turnId)
    };
    return normalized;
  }).filter((turn: ResumeTurn) => turn.items.length > 0 || isActiveResumeTurnStatus(turn.status));
}

function normalizeItems(value: ResumeContentItem[], turnId: string): ResumeContentItem[] {
  if (!Array.isArray(value)) {
    return [];
  }

  return value.map((item: ResumeContentItem, itemIndex: number) => {
    const normalized: ResumeContentItem = {
      id: normalizeString(item.id) || `${turnId}:item-${itemIndex + 1}`,

```

```
        role: normalizeString(item.role) || 'system',
        kind: normalizeString(item.kind) || 'message',
        text: normalizeString(item.text),
        status: normalizeString(item.status)
    });
    return normalized;
}).filter((item: ResumeContentItem) => item.text.length > 0);
}

function normalizeString(value: string | undefined): string {
    if (typeof value !== 'string') {
        return '';
    }

    return value.trim();
}

function normalizeStatus(value: string | ThreadStatus | undefined): string {
    if (typeof value === 'string') {
        return value;
    }

    if (value && typeof value.type === 'string') {
        return value.type;
    }

    return 'unknown';
}

export function isActiveResumeTurnStatus(status: string): boolean {
    return status === 'inProgress' || status === 'running' || status === 'queued' || status === 'in_progress';
}

function joinUrl(baseUrl: string, path: string): string {
    return `${baseUrl.replace(/\/+$/, '')}${path}`;
}

function ensureSuccess(response: GatewayHttpResponse): void {
    if (response.statusCode < 200 || response.statusCode >= 300) {
        throw new GatewayHttpError(response.statusCode);
    }
}

function ensureSuccessCode(statusCode: number): void {
    if (statusCode < 200 || statusCode >= 300) {
        throw new GatewayHttpError(statusCode);
    }
}
/*
 * AIRouter V1.0.0
 */
```

```
* 著作权人：陆豪
*/
// 固定服务器配置：App 只能连接该服务器，地址不可修改、连接不可删除。
// 注意：更改此处需重新构建并上架（证书固定 + 硬编码地址）。
export const DEFAULT_GATEWAY_SCHEME: string = 'https';
export const DEFAULT_GATEWAY_HOST: string = '8.153.174.88';
export const DEFAULT_GATEWAY_PORT: number = 8443;
export const DEFAULT_GATEWAY_DISPLAY: string = `${DEFAULT_GATEWAY_HOST}:${DEFAULT_GATEWAY_PORT}`;
export const DEFAULT_GATEWAY_NAME: string = 'AIRouter';
/*
 * AIRouter V1.0.0
 * 著作权人：陆豪
 */
import { preferences } from '@kit.ArkData';
import { common } from '@kit.AbilityKit';

const DEVICE_ID_PREFS_NAME: string = 'ai_router_device';
const DEVICE_ID_PREFS_KEY: string = 'device_id';

let cachedDeviceId: string = '';

function generateUUID(): string {
  const chars = '0123456789abcdef';
  let uuid = '';
  for (let i = 0; i < 36; i++) {
    if (i === 8 || i === 13 || i === 18 || i === 23) {
      uuid += '-';
    } else {
      const randomIndex = Math.floor(Math.random() * 16);
      uuid += chars.charAt(randomIndex);
    }
  }
  return uuid;
}

export async function getDeviceId(context: common.Context): Promise<string> {
  if (cachedDeviceId.length > 0) {
    return cachedDeviceId;
  }

  try {
    const store = await preferences.getPreferences(context, DEVICE_ID_PREFS_NAME);
    const saved = await store.get(DEVICE_ID_PREFS_KEY, '');
    if (typeof saved === 'string' && saved.trim().length > 0) {
      cachedDeviceId = saved.trim();
      return cachedDeviceId;
    }
  } catch (error) {
  }
}
```

```
const newId = generateUUID();
cachedDeviceId = newId;

try {
  const store = await preferences.getPreferences(context, DEVICE_ID_PREFS_NAME);
  await store.put(DEVICE_ID_PREFS_KEY, newId);
  await store.flush();
} catch (error) {
}

return newId;
}
/*
 * AIRouter V1.0.0
 * 著作权人: 陆豪
 */
import { huks } from '@kit.UniversalKeystoreKit';
import { BusinessError } from '@kit.BasicServicesKit';

const KEY_ALIAS: string = 'ai_router_device_auth';

function stringToUint8Array(str: string): Uint8Array {
  const result: number[] = [];
  for (let i = 0; i < str.length; i++) {
    result.push(str.charCodeAt(i) & 0xff);
  }
  return new Uint8Array(result);
}

function uint8ArrayToBase64(data: Uint8Array): string {
  const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/';
  let result = '';
  for (let i = 0; i < data.length; i += 3) {
    const b0 = data[i];
    const b1 = i + 1 < data.length ? data[i + 1] : 0;
    const b2 = i + 2 < data.length ? data[i + 2] : 0;
    result += chars.charAt(b0 >> 2);
    result += chars.charAt(((b0 & 3) << 4) | (b1 >> 4));
    result += i + 1 < data.length ? chars.charAt(((b1 & 15) << 2) | (b2 >> 6)) : '=';
    result += i + 2 < data.length ? chars.charAt(b2 & 63) : '=';
  }
  return result;
}

function wrapDerInPem(derData: Uint8Array): string {
  const b64 = uint8ArrayToBase64(derData);
  const lines: string[] = [];
  for (let i = 0; i < b64.length; i += 64) {
    lines.push(b64.substring(i, i + 64));
  }
}
```

```

    return `-----BEGIN PUBLIC KEY-----\n${lines.join('\n')}\n-----END PUBLIC KEY-----`;
}

function buildEd25519SpkiDer(rawPublicKey: Uint8Array): Uint8Array {
    // Ed25519 OID: 1.3.101.112 = 0x2b 0x65 0x70
    // SPKI structure:
    // SEQUENCE {
    //   SEQUENCE { OID 1.3.101.112 }
    //   BIT STRING { 0x00 + rawPublicKey }
    // }

    let keyBytes = rawPublicKey;
    if (keyBytes.length === 33 && keyBytes[0] === 0x00) {
        keyBytes = keyBytes.slice(1);
    }

    const algorithmId = new Uint8Array([0x30, 0x05, 0x06, 0x03, 0x2b, 0x65, 0x70]);

    const bitStringValue = new Uint8Array(1 + keyBytes.length);
    bitStringValue[0] = 0x00;
    bitStringValue.set(keyBytes, 1);

    const bitString = new Uint8Array(2 + bitStringValue.length);
    bitString[0] = 0x03;
    bitString[1] = bitStringValue.length;
    bitString.set(bitStringValue, 2);

    const contentLen = algorithmId.length + bitString.length;
    const sequence = new Uint8Array(2 + contentLen);
    sequence[0] = 0x30;
    sequence[1] = contentLen;
    sequence.set(algorithmId, 2);
    sequence.set(bitString, 2 + algorithmId.length);

    return sequence;
}

function getGenerateProperties(): huks.HuksParam[] {
    const properties: huks.HuksParam[] = [
        {
            tag: huks.HuksTag.HUKS_TAG_ALGORITHM,
            value: huks.HuksKeyAlg.HUKS_ALG_ED25519
        },
        {
            tag: huks.HuksTag.HUKS_TAG_PURPOSE,
            value: huks.HuksKeyPurpose.HUKS_KEY_PURPOSE_SIGN | huks.HuksKeyPurpose.HUKS_KEY_PURPOSE_VERIFY
        },
        {
            tag: huks.HuksTag.HUKS_TAG_KEY_SIZE,
            value: huks.HuksKeySize.HUKS_CURVE25519_KEY_SIZE_256
        }
    ];
}

```

```
    }
  ];
  return properties;
}

function getSignProperties(): huks.HuksParam[] {
  const properties: huks.HuksParam[] = [
    {
      tag: huks.HuksTag.HUKS_TAG_ALGORITHM,
      value: huks.HuksKeyAlg.HUKS_ALG_ED25519
    },
    {
      tag: huks.HuksTag.HUKS_TAG_PURPOSE,
      value: huks.HuksKeyPurpose.HUKS_KEY_PURPOSE_SIGN
    }
  ];
  return properties;
}

async function generateKey(): Promise<void> {
  const options: huks.HuksOptions = {
    properties: getGenerateProperties()
  };
  try {
    await huks.generateKeyItem(KEY_ALIAS, options);
  } catch (error) {
    throw new Error('Failed to generate device key');
  }
}

async function signData(data: Uint8Array): Promise<Uint8Array> {
  const options: huks.HuksOptions = {
    properties: getSignProperties(),
    inData: data
  };
  try {
    const sessionResult = await huks.initSession(KEY_ALIAS, options);
    const finishResult = await huks.finishSession(sessionResult.handle, options);
    return finishResult.outData as Uint8Array;
  } catch (error) {
    throw new Error('Failed to sign data');
  }
}

async function exportPublicKeyDer(): Promise<Uint8Array> {
  const emptyOptions: huks.HuksOptions = {
    properties: []
  };
  try {
    const result = await huks.exportKeyItem(KEY_ALIAS, emptyOptions);
  }
}
```

```
        return result.outData as Uint8Array;
    } catch (error) {
        throw new Error('Failed to export device public key');
    }
}

export interface DeviceKeyPair {
    publicKeyPem: string;
}

let cachedKeyPair: DeviceKeyPair | null = null;

export async function ensureDeviceKeyPair(): Promise<DeviceKeyPair> {
    if (cachedKeyPair !== null) {
        return cachedKeyPair;
    }

    let rawKey: Uint8Array;
    try {
        rawKey = await exportPublicKeyDer();
    } catch {
        await generateKey();
        rawKey = await exportPublicKeyDer();
    }

    const spkiDer = buildEd25519SpkiDer(rawKey);
    const publicKeyPem = wrapDerInPem(spkiDer);
    cachedKeyPair = { publicKeyPem };
    return cachedKeyPair;
}

export async function signChallenge(challenge: string): Promise<string> {
    const data = stringToUint8Array(challenge);
    const sig = await signData(data);
    return uint8ArrayToBase64(sig);
}

/*
 * AIRouter V1.0.0
 * 著作权人: 陆豪
 */
import { huks } from '@kit.UniversalKeystoreKit';
import { buffer, util } from '@kit.ArkTS';

const CONNECTION_KEY_ALIAS: string = 'codex_router_connection_store_key';
const ENCRYPTED_STORE_PREFIX: string = 'ENC1:';
const GCM_TAG_LENGTH: number = 16;
const GCM_NONCE_LENGTH: number = 12;

function connectionKeyParams(): huks.HuksParam[] {
    return [
```

```

    { tag: huks.HuksTag.HUKS_TAG_ALGORITHM, value: huks.HuksKeyAlg.HUKS_ALG_AES },
    { tag: huks.HuksTag.HUKS_TAG_KEY_SIZE, value: huks.HuksKeySize.HUKS_AES_KEY_SIZE_256 },
    {
      tag: huks.HuksTag.HUKS_TAG_PURPOSE,
      value: huks.HuksKeyPurpose.HUKS_KEY_PURPOSE_ENCRYPT | huks.HuksKeyPurpose.HUKS_KEY_PURPOSE_DECRYPT
    },
    { tag: huks.HuksTag.HUKS_TAG_PADDING, value: huks.HuksKeyPadding.HUKS_PADDING_NONE },
    { tag: huks.HuksTag.HUKS_TAG_BLOCK_MODE, value: huks.HuksCipherMode.HUKS_MODE_GCM }
  ];
}

function cipherParams(purpose: huks.HuksKeyPurpose, nonce: Uint8Array | null, tag: Uint8Array | null):
huks.HuksParam[] {
  const params: huks.HuksParam[] = [
    { tag: huks.HuksTag.HUKS_TAG_ALGORITHM, value: huks.HuksKeyAlg.HUKS_ALG_AES },
    { tag: huks.HuksTag.HUKS_TAG_KEY_SIZE, value: huks.HuksKeySize.HUKS_AES_KEY_SIZE_256 },
    { tag: huks.HuksTag.HUKS_TAG_PURPOSE, value: purpose },
    { tag: huks.HuksTag.HUKS_TAG_PADDING, value: huks.HuksKeyPadding.HUKS_PADDING_NONE },
    { tag: huks.HuksTag.HUKS_TAG_BLOCK_MODE, value: huks.HuksCipherMode.HUKS_MODE_GCM }
  ];

  if (nonce !== null) {
    params.push({ tag: huks.HuksTag.HUKS_TAG_NONCE, value: nonce });
  }

  if (tag !== null) {
    params.push({ tag: huks.HuksTag.HUKS_TAG_AE_TAG, value: tag });
  }

  return params;
}

async function ensureConnectionKey(): Promise<void> {
  try {
    await huks.generateKeyItem(CONNECTION_KEY_ALIAS, { properties: connectionKeyParams() });
  } catch (error) {
  }
}

function utf8Encode(value: string): Uint8Array {
  const textEncoder = new util.TextEncoder();
  return textEncoder.encodeInto(value);
}

function utf8Decode(value: Uint8Array): string {
  const textDecoder = new util.TextDecoder('utf-8');
  return textDecoder.decodeToString(value);
}

function copyBufferToBytes(value: buffer.Buffer): Uint8Array {
  const bytes = new Uint8Array(value.length);

```

```
    for (let i = 0; i < value.length; i++) {
        bytes[i] = value[i];
    }
    return bytes;
}

export function isEncryptedConnectionStore(storedValue: string): boolean {
    return storedValue.startsWith(ENCRYPTED_STORE_PREFIX);
}

export async function encryptConnectionStore(plainText: string): Promise<string> {
    try {
        const options: huks.HuksOptions = {
            properties: cipherParams(huks.HuksKeyPurpose.HUKS_KEY_PURPOSE_ENCRYPT, null, null),
            inData: utf8Encode(plainText)
        };

        await ensureConnectionKey();
        const handle = await huks.initSession(CONNECTION_KEY_ALIAS, options);
        const finish = await huks.finishSession(handle.handle, options);
        const outData = finish.outData;

        if (outData === undefined || outData.length === 0) {
            throw new Error('Failed to encrypt connection store');
        }

        return ENCRYPTED_STORE_PREFIX + buffer.from(outData).toString('base64');
    } catch (error) {
        throw new Error('Failed to encrypt connection store');
    }
}

export async function decryptConnectionStore(storedValue: string): Promise<string | null> {
    if (!isEncryptedConnectionStore(storedValue)) {
        return null;
    }

    let blob: Uint8Array;
    try {
        const decoded = buffer.from(storedValue.substring(ENCRYPTED_STORE_PREFIX.length), 'base64');
        blob = copyBufferToBytes(decoded);
    } catch (error) {
        return null;
    }

    const minimumLength: number = GCM_NONCE_LENGTH + GCM_TAG_LENGTH;
    if (blob.length < minimumLength) {
        return null;
    }
}
```

```
const nonce = blob.subarray(blob.length - GCM_NONCE_LENGTH);
const tag = blob.subarray(blob.length - GCM_NONCE_LENGTH - GCM_TAG_LENGTH, blob.length - GCM_NONCE_LENGTH);
const cipher = blob.subarray(0, blob.length - GCM_NONCE_LENGTH - GCM_TAG_LENGTH);
const options: huks.HuksOptions = {
  properties: cipherParams(huks.HuksKeyPurpose.HUKS_KEY_PURPOSE_DECRYPT, nonce, tag),
  inData: cipher
};

try {
  const handle = await huks.initSession(CONNECTION_KEY_ALIAS, options);
  const finish = await huks.finishSession(handle.handle, options);
  const outData = finish.outData;

  if (outData === undefined || outData.length === 0) {
    return null;
  }

  return utf8Decode(outData);
} catch (error) {
  return null;
}
}
}
/*
 * AIRouter V1.0.0
 * 著作权人: 陆豪
 */
export type ConnectionStatus = 'online' | 'offline' | 'needs_login' | 'unknown';
export type ConnectionSwitchIntent = 'turn_on_and_show_resumes' | 'turn_off' | 'require_login';
export type ResumeFeedbackAction = 'none' | 'relogin';
export type ConnectionAgent = 'OpenCode';

export interface SavedConnection {
  id: string;
  name: string;
  agent: ConnectionAgent;
  host: string;
  port: number;
  scheme: string;
  username: string;
  lastUsedAt: string;
  status: ConnectionStatus;
  accessTokenRef: string;
  refreshTokenRef: string;
  privateKeyRef: string;
}

export interface NormalizedServerAddress {
  scheme: string;
  host: string;
  port: number;
}
```

```
    baseUrl: string;
}

export interface EditableConnectionFields {
  id: string;
  name: string;
  agent: ConnectionAgent;
  host: string;
  port: number;
  scheme: string;
  username: string;
  lastUsedAt: string;
  accessTokenRef?: string;
  refreshTokenRef?: string;
  privateKeyRef?: string;
}

export interface ConnectionFormErrors {
  address: string;
  privateKey: string;
  hasError: boolean;
}

export interface ConnectionCarouselSelection {
  index: number;
  id: string;
}

// 固定单服务器：只允许一条连接。
export const MAX_CONNECTIONS: number = 1;

const DEFAULT_SCHEME: string = 'https';
const DEFAULT_PORT: number = 8443;
const RESUME_CONTENT_BASE_FONT_SCALE: number = 0.9;

export function normalizeServerAddress(input: string): NormalizedServerAddress {
  let trimmedInput = input.trim();
  let scheme = DEFAULT_SCHEME;
  let address = trimmedInput;

  const schemeSeparatorIndex = trimmedInput.indexOf('://');
  if (schemeSeparatorIndex >= 0) {
    scheme = trimmedInput.substring(0, schemeSeparatorIndex);
    address = trimmedInput.substring(schemeSeparatorIndex + 3);
  }

  let host = address;
  let port = DEFAULT_PORT;
  const portSeparatorIndex = address.lastIndexOf(':');
  if (portSeparatorIndex > -1) {

```

```
    const portText = address.substring(portSeparatorIndex + 1);
    const parsedPort = Number(portText);
    if (parsedPort === parsedPort && parsedPort > 0 && parsedPort <= 65535) {
        host = address.substring(0, portSeparatorIndex);
        port = parsedPort;
    }
}

return {
    scheme: scheme,
    host: host,
    port: port,
    baseUrl: scheme + '://' + host + ':' + port.toString()
};
}

export function resolveConnectionName(
    customName: string,
    sessionName: string,
    username: string,
    host: string
): string {
    const normalizedCustomName = customName.trim();
    if (normalizedCustomName.length > 0) {
        return normalizedCustomName;
    }

    const normalizedHost = host.trim();
    if (normalizedHost.length > 0) {
        return normalizedHost;
    }

    const normalizedSessionName = sessionName.trim();
    if (normalizedSessionName.length > 0) {
        return normalizedSessionName;
    }

    return username.trim();
}

export function resolveConnectionDisplayName(
    connections: SavedConnection[],
    id: string,
    fallbackName: string
): string {
    const found = connections.find((item: SavedConnection) => item.id === id);
    if (found !== undefined) {
        return found.name;
    }
}
```

```
    return fallbackName;
}

export function resolveConnectionCardTitle(connection: SavedConnection): string {
    return connection.host.trim();
}

export function canAddConnection(connections: SavedConnection[]): boolean {
    return connections.length < MAX_CONNECTIONS;
}

function normalizeConnectionIp(ip: string): string {
    return ip.trim().toLowerCase();
}

export function generateSavedConnectionId(ip: string): string {
    return `OpenCode-${normalizeConnectionIp(ip)}`;
}

export function hasConnectionWithSameEndpoint(
    connections: SavedConnection[],
    scheme: string,
    host: string,
    port: number,
    ignoredConnectionId: string
): boolean {
    const targetId = generateSavedConnectionId(host);

    return connections.some((item: SavedConnection) => {
        if (ignoredConnectionId.length > 0 && item.id === ignoredConnectionId) {
            return false;
        }

        return generateSavedConnectionId(item.host) === targetId;
    });
}

export function sortConnectionsByLastUsed(connections: SavedConnection[]): SavedConnection[] {
    return connections.slice().sort((left: SavedConnection, right: SavedConnection) => {
        return Date.parse(right.lastUsedAt) - Date.parse(left.lastUsedAt);
    });
}

export function resolveCarouselIndex(currentIndex: number, itemCount: number): number {
    if (itemCount <= 0) {
        return 0;
    }

    if (currentIndex < 0) {
        return 0;
    }
}
```

```
'write', 'edit', 'patch', 'apply_patch', 'multiedit', 'create', 'write_file', 'str_replace_editor'
]);
```

// shell 类工具：需从命令/输出里扫文件路径（bash 直接写文件不会被上面识别）。

```
const SHELL_TOOLS = new Set(['bash', 'shell', 'sh', 'console', 'terminal']);
```

// 会话兜底命名：取首条消息前 N 个字符。

```
const SESSION_TITLE_MAX_LENGTH = 20;
```

// 会被登记的产出文件扩展名（与 file-service 白名单一致）。

```
/**
 * 从会话消息 parts 中提取“产出/改动的文件路径”。
 * 用于把文件按会话归属到设备（不依赖 opencode 的目录隔离）。
 *
 * 覆盖：
 * - `file` part 的 filename/path
 * - 写文件类工具（write/edit/patch...）input/metadata 中含 file/path 的字段
 * - bash/shell 工具的命令与输出中以白名单扩展名结尾的路径（覆盖 `cat > x.docx`、脚本产出等）
 */
```

```
export function extractFilePathsFromMessages(messages) {
  if (!Array.isArray(messages)) {
    return [];
  }
}
```

```
const paths = new Set();
const add = (value) => {
  const path = normalizeString(value);
  if (path.length > 0) {
    paths.add(path);
  }
};
```

```
for (const message of messages) {
  const parts = Array.isArray(message?.parts) ? message.parts : [];
  for (const part of parts) {
    const type = normalizeString(part?.type);
    if (type === 'file') {
      add(part?.filename);
      add(part?.path);
      continue;
    }
    if (type !== 'tool') {
      continue;
    }
  }
}
```

```
const toolName = normalizeString(part?.tool).toLowerCase();
const state = part?.state ?? {};
```

```
if (FILE_PRODUCING_TOOLS.has(toolName)) {
  // 不同工具/版本字段名不一（filePath / file_path / path / filename ...）。
}
```

```

        for (const value of pickPathLikeValues(state.input)) {
            add(value);
        }
        for (const value of pickPathLikeValues(state.metadata)) {
            add(value);
        }
    }
}

// shell 类工具: 从命令/输出里扫出以白名单扩展名结尾的路径 (覆盖 bash/脚本产物)。
if (SHELL_TOOLS.has(toolName)) {
    for (const value of collectStringValue(state.input)) {
        for (const token of extractPathTokens(value)) {
            add(token);
        }
    }
    for (const value of collectStringValue(state.metadata)) {
        for (const token of extractPathTokens(value)) {
            add(token);
        }
    }
    for (const token of extractPathTokens(normalizeString(state.output))) {
        add(token);
    }
}
}
}

return Array.from(paths);
}

/** 从 `~/session/:id/diff` 的返回中提取文件路径 (字段名未知, 做通用兜底)。 */
export function extractFilePathsFromDiff(diff) {
    if (!Array.isArray(diff)) {
        return [];
    }
    const paths = new Set();
    const add = (value) => {
        const path = normalizeString(value);
        if (path.length > 0) {
            paths.add(path);
        }
    };
    for (const entry of diff) {
        if (!entry || typeof entry !== 'object') {
            continue;
        }
        add(entry.file);
        add(entry.path);
        add(entry.filename);
        for (const value of pickPathLikeValues(entry)) {

```

```

        add(value);
    }
    for (const value of collectStringValue(entry)) {
        for (const token of extractPathTokens(value)) {
            add(token);
        }
    }
}
return Array.from(paths);
}

/** 收集对象中所有字符串值（一层）。 */
function collectStringValue(source) {
    if (!source || typeof source !== 'object') {
        return [];
    }
    return Object.values(source).filter((value) => typeof value === 'string');
}

/** 从任意文本里提取“以白名单扩展名结尾”的路径 token。 */
function extractPathTokens(text) {
    const value = normalizeString(text);
    if (value.length === 0) {
        return [];
    }
    const tokens = value.split(/[\s"'`|<>,;(){}]+/);
    const results = [];
    for (const raw of tokens) {
        const token = raw.replace(/[\.,;]+$/, '');
        if (token.length > 0 && token.length < 1024 && isAllowedFile(token)) {
            results.push(token);
        }
    }
    return results;
}

function pickPathLikeValues(source) {
    if (!source || typeof source !== 'object') {
        return [];
    }
    const values = [];
    for (const [key, value] of Object.entries(source)) {
        if (typeof value === 'string' && value.length > 0 && value.length < 1024 && /file|path/i.test(key)) {
            values.push(value);
        }
    }
    return values;
}

export class OpenCodeServerClient {

```

```

    constructor(options = {}) {
      this.url = stripTrailingSlash(options.url ?? process.env.OPENCODE_SERVER_URL ?? DEFAULT_OPENCODE_SERVER_URL);
      this.command = options.command ?? process.env.OPENCODE_COMMAND ?? DEFAULT_OPENCODE_COMMAND;
      this.workdir = normalizeString(options.workdir ?? process.env.OPENCODE_WORKDIR ?? process.env.HOME ??
homedir()) ||
      process.cwd();
      this.process = null;
      this.fetchImpl = options.fetchImpl ?? fetch;
      this.spawnServer = options.spawnServer ?? spawn;
      this.platform = options.platform ?? process.platform;
      this.runCommand = options.runCommand ?? ((args) => runCommand(this.spawnServer, this.command, args, {
        cwd: this.workdir,
        platform: this.platform
      }));
      this.now = options.now ?? Date.now;
      this.stderr = '';
      this.startupError = null;
      this.sessionsCacheTtlMs = options.sessionsCacheTtlMs ?? SESSIONS_CACHE_TTL_MS;
      this.sessionsCache = new Map();
    }

    async listResumes(options = {}) {
      const limit = options.limit ?? 20;
      const normalizedLimit = Number.isFinite(limit) && limit > 0 ? limit : 20;
      const rows = await this.loadSessionRows(normalizedLimit);
      const sessions = rows.map(mapOpenCodeDbSessionToSession);
      return sessions
        .filter((session) => !session?.time?.archived)
        .sort((left, right) => Number(right?.time?.updated || 0) - Number(left?.time?.updated || 0))
        .slice(0, normalizedLimit)
        .map(mapOpenCodeSessionToResumeItem);
    }

    /** 带短 TTL 缓存地读取会话列表, 避免每次请求都 spawn `opencode db`。 */
    async loadSessionRows(limit) {
      const cached = this.sessionsCache.get(limit);
      const now = this.now();
      if (cached && now - cached.at < this.sessionsCacheTtlMs) {
        return cached.rows;
      }
      const rows = await this.querySessions(limit);
      this.sessionsCache.set(limit, { at: now, rows });
      return rows;
    }

    invalidateSessionsCache() {
      this.sessionsCache.clear();
    }

    async readResume(options = {}) {
      const threadId = requireThreadId(options.threadId);

```

```

const directory = normalizeString(options.directory);
const session = await this.request(`/session/${encodeURIComponent(threadId)}`, { directory });
const messages = await this.request(`/session/${encodeURIComponent(threadId)}/message`, { directory });
const messageFiles = extractFilePathsFromMessages(messages);
// 会话 diff 能覆盖 bash/脚本产出的文件; 失败时静默降级为仅用消息解析结果。
const diffFiles = await this.readSessionDiff(threadId, directory);
return {
  ...mapOpenCodeSessionToResumeSession(session, messages),
  files: Array.from(new Set([...messageFiles, ...diffFiles]))
};
}

async readSessionDiff(threadId, directory) {
  try {
    const diff = await this.request(`/session/${encodeURIComponent(threadId)}/diff`, {
      directory,
      timeout: 5000
    });
    return extractFilePathsFromDiff(diff);
  } catch (error) {
    return [];
  }
}

async startResume(options = {}) {
  // directory 决定 opencode 的项目/工作目录: 不同设备各自的工作区。
  const directory = normalizeString(options.directory) || normalizeString(options.cwd);
  // 不复用目录当标题: 让 opencode 按首句自动命名 (未自动命名时由 sendResumeMessage 兜底)。
  const session = await this.request('/session', {
    method: 'POST',
    body: {},
    directory: directory !== '~' ? directory : ''
  });
  this.invalidateSessionsCache();
  return mapOpenCodeSessionToResumeSession(session, []);
}

async sendResumeMessage(options = {}) {
  const threadId = requireThreadId(options.threadId);
  const message = normalizeString(options.message);
  if (message.length === 0) {
    throw new Error('message is required');
  }

  // prompt_async: 发送即返回 204, 不等待生成完成; 结果由客户端轮询 /resume 获取 (不阻塞、不取消生成)。
  await this.request(`/session/${encodeURIComponent(threadId)}/prompt_async`, {
    method: 'POST',
    body: {
      parts: [
        {

```

```

        type: 'text',
        text: message
      }
    ],
  },
  directory: normalizeString(options.directory)
});

const session = await this.readResume({ threadId, directory: options.directory });

// 兜底命名: 若 opencode 还没按首句命名 (标题仍为空/就是 session id), 用本条消息前 20 字命名。
if (isSessionTitleUnset(session.title, threadId)) {
  const title = deriveSessionTitle(message);
  if (title.length > 0) {
    try {
      await this.renameResume({ threadId, name: title, directory: normalizeString(options.directory) });
      session.title = title;
    } catch (error) {
      console.error(`[opencode] 会话兜底命名失败: ${error instanceof Error ? error.message : error}`);
    }
  }
}

return session;
}

async archiveResume(options = {}) {
  const threadId = requireThreadId(options.threadId);
  const now = Number(this.now());
  await this.runCommand([
    'db',
    `update session set time_archived = ${now}, time_updated = ${now} where id = '${escapeSqlLiteral(threadId)}'`
  ]);
  this.invalidateSessionsCache();
}

async renameResume(options = {}) {
  const threadId = requireThreadId(options.threadId);
  const name = normalizeString(options.name);
  if (name.length === 0) {
    throw new Error('name is required');
  }

  await this.request(`/session/${encodeURIComponent(threadId)}`, {
    method: 'PATCH',
    body: {
      title: name
    },
    directory: normalizeString(options.directory)
  });
}

```

```

    this.invalidateSessionsCache();
  }

  async deleteResume(options = {}) {
    const threadId = requireThreadId(options.threadId);
    await this.runCommand(['session', 'delete', threadId]);
    this.invalidateSessionsCache();
  }

  async querySessions(limit) {
    const stdout = await this.runCommand([
      'db',
      `select id, title, slug, directory, path, time_created, time_updated, time_archived from session where
time_archived is null order by time_updated desc limit ${limit}`,
      '--format',
      'json'
    ]);
    const text = normalizeString(stdout);
    return text.length > 0 ? JSON.parse(text) : [];
  }

  async request(path, options = {}) {
    await this.ensureReady();

    // opencode 按 `directory` 解析项目/工作目录; 用它把不同设备隔离到各自工作区。
    let target = `${this.url}${path}`;
    if (normalizeString(options.directory).length > 0) {
      const url = new URL(target);
      url.searchParams.set('directory', normalizeString(options.directory));
      target = url.toString();
    }

    const timeoutMs = options.timeout ?? 300000;
    const response = await this.fetchImpl(target, {
      method: options.method ?? 'GET',
      headers: options.body !== undefined ? {
        'content-type': 'application/json'
      } : undefined,
      body: options.body !== undefined ? JSON.stringify(options.body) : undefined,
      signal: AbortSignal.timeout(timeoutMs)
    });
    if (!response?.ok) {
      throw new Error(`opencode server request failed: ${response?.status ?? 'no response'}`);
    }

    const text = await response.text();
    return text.trim().length > 0 ? JSON.parse(text) : {};
  }

  async ensureReady() {
    if (await this.isReady()) {

```

```
        return;
    }

    await this.startServer();
    const startedAt = Date.now();
    while (Date.now() - startedAt < SERVER_READY_TIMEOUT_MS) {
        if (await this.isReady()) {
            return;
        }
        if (!this.process) {
            if (this.startupError) {
                throw this.startupError;
            }
            throw new Error(this.describeStartupError('opencode serve exited before ready'));
        }
        await wait(SERVER_READY_INTERVAL_MS);
    }

    throw new Error(this.describeStartupError('Timed out waiting for opencode serve'));
}

async isReady() {
    try {
        const response = await this.fetchImpl(`${this.url}/session`);
        return Boolean(response?.ok);
    } catch (_error) {
        return false;
    }
}

async startServer() {
    if (this.process) {
        return;
    }

    const listenUrl = new URL(this.url);
    const spawnOptions = {
        stdio: ['ignore', 'ignore', 'pipe'],
        cwd: this.workdir
    };
    if (this.platform === 'win32') {
        spawnOptions.shell = true;
        spawnOptions.windowsHide = true;
    }

    const child = this.spawnServer(this.command, [
        'serve',
        '--hostname',
        listenUrl.hostname,
        '--port',
```

```
listenUrl.port || '4096',
  '--print-logs'
], spawnOptions);
this.process = child;
this.startupError = null;

child.stderr?.on('data', (chunk) => {
  this.stderr += chunk.toString();
  if (this.stderr.length > 4000) {
    this.stderr = this.stderr.slice(-4000);
  }
});

child.once('error', (error) => {
  this.process = null;
  this.startupError = error;
});

child.once('exit', () => {
  this.process = null;
});
}

describeStartupError(message) {
  const stderr = this.stderr.trim();
  if (stderr.length === 0) {
    return message;
  }

  return `${message}\n${stderr}`;
}

function mapOpenCodeDbSessionToSession(row) {
  return {
    id: row?.id,
    title: row?.title,
    slug: row?.slug,
    directory: row?.directory,
    path: row?.path,
    time: {
      created: row?.time_created,
      updated: row?.time_updated,
      archived: row?.time_archived
    }
  };
}

function mapOpenCodePartToResumeContent(messageId, role, part, partIndex) {
  const partId = normalizeString(part?.id) || `part-${partIndex + 1}`;
```

```

const type = normalizeString(part?.type) || 'part';

if (type === 'text') {
  if (role !== 'user' && role !== 'assistant') {
    return null;
  }

  return resumeContent(`${messageId}:${partId}`, role, 'message', part?.text, '');
}

// 工具调用 / 文件产物也映射为内容项，让“生成文件/调用工具”这类长任务在客户端有进度可看。
if (type === 'tool') {
  const toolName = normalizeString(part?.tool) || 'tool';
  const status = normalizeString(part?.state?.status);
  const text = status.length > 0 ? `${toolName} • ${status}` : toolName;
  return resumeContent(`${messageId}:${partId}`, 'tool', 'tool', text, status);
}

if (type === 'file') {
  const filename = normalizeString(part?.filename) || normalizeString(part?.path) || 'file';
  // 只展示文件名+后缀，不展示完整路径。
  return resumeContent(`${messageId}:${partId}`, 'tool', 'file', baseName(filename), '');
}

return null;
}

function resumeContent(id, role, kind, text, status) {
  const normalizedText = truncateText(normalizePartText(text));
  if (normalizedText.length === 0) {
    return null;
  }

  return {
    id,
    role,
    kind,
    text: normalizedText,
    status
  };
}

function normalizePartText(value) {
  if (typeof value === 'string') {
    return value.trim();
  }

  if (Array.isArray(value)) {
    return value.map(normalizePartText).filter(Boolean).join('\n');
  }
}

```

```
    if (value && typeof value === 'object') {
      return JSON.stringify(value);
    }

    return '';
  }

  function normalizeRole(role) {
    if (role === 'assistant' || role === 'user' || role === 'tool' || role === 'system') {
      return role;
    }

    return 'system';
  }

  function resolveMessageStatus(info) {
    if (info?.error) {
      return 'error';
    }

    if (info?.role === 'assistant' && !info?.time?.completed) {
      return 'running';
    }

    return 'completed';
  }

  function isActiveStatus(status) {
    return status === 'running' || status === 'inProgress' || status === 'in_progress' || status === 'queued';
  }

  function timestampToIso(value) {
    const timestamp = Number(value || 0);
    return timestamp > 0 ? new Date(timestamp).toISOString() : '';
  }

  function truncateText(value) {
    if (value.length <= MAX_PART_TEXT_LENGTH) {
      return value;
    }

    return `${value.slice(0, MAX_PART_TEXT_LENGTH)}\n...`;
  }

  function normalizeString(value) {
    return typeof value === 'string' ? value.trim() : '';
  }

  /** 只取路径的文件名部分（兼容 `/`、`\`）。 */
```

```

function baseName(value) {
  const text = normalizeString(value).replace(/\\/g, '/');
  const index = text.lastIndexOf('/');
  return index >= 0 ? text.slice(index + 1) : text;
}

/** 会话标题是否“未命名”（空 / 就是 session id / 形如 ses_xxx）。 */
function isSessionTitleUnset(title, threadId) {
  const value = normalizeString(title);
  if (value.length === 0) {
    return true;
  }
  if (value === normalizeString(threadId)) {
    return true;
  }
  return /^ses[_-]/i.test(value);
}

/** 兜底标题：取消息前 N 个字符（换行保留）。 */
function deriveSessionTitle(message) {
  const text = typeof message === 'string' ? message.replace(/\r\n/g, '\n').trim() : '';
  return Array.from(text).slice(0, SESSION_TITLE_MAX_LENGTH).join('');
}

function requireThreadId(threadId) {
  const normalized = normalizeString(threadId);
  if (normalized.length === 0) {
    throw new Error('threadId is required');
  }

  return normalized;
}

function stripTrailingSlash(value) {
  return normalizeString(value).replace(/\/+$/, '');
}

function escapeSqlLiteral(value) {
  return normalizeString(value).replace(/'/g, '');
}

function runCommand(spawnCommand, command, args, options = {}) {
  return new Promise((resolve, reject) => {
    const spawnOptions = {
      cwd: options.cwd,
      stdio: ['ignore', 'pipe', 'pipe']
    };
    if (options.platform === 'win32') {
      spawnOptions.shell = true;
      spawnOptions.windowsHide = true;
    }
  });
}

```

```
    }

    const child = spawnCommand(command, args, spawnOptions);
    let stdout = '';
    let stderr = '';

    child.stdout?.on('data', (chunk) => {
      stdout += chunk.toString();
    });
    child.stderr?.on('data', (chunk) => {
      stderr += chunk.toString();
    });
    child.once('error', reject);
    child.once('exit', (code) => {
      if (code === 0) {
        resolve(stdout);
        return;
      }
      reject(new Error(`opencode command failed (${code}): ${stderr.trim()}`));
    });
  });
}

function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}
/*
 * AIRouter V1.0.0
 * 著作权人: 陆豪
 */
const CLEANUP_INTERVAL_MS = 60 * 1000;

export class RateLimiter {
  #windowMs;
  #maxRequests;
  #clients;
  #cleanupTimer;

  constructor(windowMs, maxRequests) {
    this.#windowMs = windowMs;
    this.#maxRequests = maxRequests;
    this.#clients = new Map();

    this.#cleanupTimer = setInterval(() => this.#cleanup(), CLEANUP_INTERVAL_MS);
    if (this.#cleanupTimer.unref) {
      this.#cleanupTimer.unref();
    }
  }
}
```

```
check(key) {
  const now = Date.now();
  const entry = this.#clients.get(key);

  if (!entry || now >= entry.resetAt) {
    this.#clients.set(key, { count: 1, resetAt: now + this.#windowMs });
    return true;
  }

  if (entry.count >= this.#maxRequests) {
    return false;
  }

  entry.count += 1;
  return true;
}

reset(key) {
  this.#clients.delete(key);
}

#cleanup() {
  const now = Date.now();
  for (const [key, entry] of this.#clients) {
    if (now >= entry.resetAt) {
      this.#clients.delete(key);
    }
  }
}

export function getClientIp(request) {
  const forwarded = request.headers['x-forwarded-for'];
  if (forwarded) {
    return forwarded.split(',')[0].trim();
  }

  const realIp = request.headers['x-real-ip'];
  if (realIp) {
    return realIp.trim();
  }

  return request.socket?.remoteAddress ?? '';
}

const FIFTEEN_MINUTES = 15 * 60 * 1000;
const FIVE_MINUTES = 5 * 60 * 1000;
const ONE_HOUR = 60 * 60 * 1000;

export const registerLimiter = new RateLimiter(FIFTEEN_MINUTES, 5);
```

```
export const activateLimiter = new RateLimiter(FIVE_MINUTES, 3);
export const challengeLimiter = new RateLimiter(FIFTEEN_MINUTES, 10);
export const verifyLimiter = new RateLimiter(FIFTEEN_MINUTES, 5);
export const refreshLimiter = new RateLimiter(FIFTEEN_MINUTES, 10);
export const reregisterLimiter = new RateLimiter(ONE_HOUR, 3);
/*
 * AIRouter V1.0.0
 * 著作权人: 陆豪
 */
import http from 'node:http';
import https from 'node:https';
import fs from 'node:fs';
import path from 'node:path';
import { randomUUID } from 'node:crypto';
import { OpenCodeServerClient } from './opencode-server.mjs';
import {
  verifyAccessToken,
  ensureDeviceRegistrationCode,
  createActivationChallenge,
  verifyActivation,
  createLoginChallenge,
  verifyLogin,
  refreshAccessToken,
  reregisterDevice,
  logoutDevice,
  cleanupExpiredNonces,
} from './auth.mjs';
import {
  registerLimiter,
  activateLimiter,
  challengeLimiter,
  verifyLimiter,
  refreshLimiter,
  reregisterLimiter,
  getClientIp,
} from './rate-limiter.mjs';
import {
  resolveFilesRoot,
  deviceWorkspace,
  isAllowedFile,
  mimeTypeForFile,
} from './file-service.mjs';
import {
  saveDeviceSession,
  getDeviceSessionOwner,
  listDeviceSessions,
  deleteDeviceSession,
  upsertSessionFile,
  listSessionFileRecords,
} from './db.mjs';
```

```
const DEFAULT_HOST = '0.0.0.0';
const DEFAULT_PORT = 8443;

const PROJECT = {
  id: 'codex-router',
  name: 'AIRouter',
  defaultBranch: 'master',
  status: 'ready',
  permissions: ['task:create', 'task:read', 'task:write', 'diff:read', 'approval:review']
};

function jsonResponse(response, statusCode, body) {
  response.writeHead(statusCode, {
    'content-type': 'application/json; charset=utf-8',
    'access-control-allow-origin': '*',
    'access-control-allow-methods': 'GET,POST,DELETE,OPTIONS',
    'access-control-allow-headers': 'authorization,content-type'
  });
  response.end(JSON.stringify(body));
}

function ErrorResponse(response, statusCode, code, message) {
  jsonResponse(response, statusCode, {
    error: { code, message, requestId: `req_${randomUUID()}` }
  });
}

function readJson(request) {
  return new Promise((resolve, reject) => {
    let raw = '';
    request.setEncoding('utf8');
    request.on('data', (chunk) => { raw += chunk; });
    request.on('end', () => {
      if (raw.trim().length === 0) { resolve({}); return; }
      try { resolve(JSON.parse(raw)); } catch (e) { reject(e); }
    });
    request.on('error', reject);
  });
}

function bearerToken(request) {
  const header = request.headers.authorization ?? '';
  if (!header.startsWith('Bearer ')) return '';
  return header.substring('Bearer '.length);
}

function normalizeResumeCwd(value) {
  const cwd = typeof value === 'string' ? value.trim() : '';
  return cwd.length > 0 ? cwd : '~';
}
```

```
}

function requireAuth(request, response) {
  const token = bearerToken(request);
  if (!token) {
    errorResponse(response, 401, 'token_missing', 'Authorization token required');
    return null;
  }
  const payload = verifyAccessToken(token);
  if (!payload) {
    errorResponse(response, 401, 'token_expired', 'Token is expired or invalid');
    return null;
  }
  return { deviceId: payload.sub };
}

/**
 * 会话归属校验：只有创建该会话的设备才能访问。非归属方返回 404（不泄漏会话是否存在）。
 */
function requireThreadOwnership(response, deviceId, threadId) {
  const owner = getDeviceSessionOwner(threadId);
  if (!owner || owner !== deviceId) {
    errorResponse(response, 404, 'session_not_found', 'Session not found');
    return false;
  }
  return true;
}

/**
 * 把会话产生的文件路径登记到 session_files（按设备归属）。
 * 只登记位于 filesRoot 之下、扩展名白名单、且当前存在的文件。
 */
function recordSessionFiles(deviceId, threadId, paths) {
  if (!deviceId || !threadId || !Array.isArray(paths) || paths.length === 0) {
    return;
  }
  const filesRoot = resolveFilesRoot();
  if (!filesRoot) {
    return;
  }
  const root = path.resolve(filesRoot);
  for (const raw of paths) {
    const candidate = typeof raw === 'string' ? raw.trim() : '';
    if (candidate.length === 0) {
      continue;
    }
    const absolute = path.resolve(path.isAbsolute(candidate) ? candidate : path.join(root, candidate));
    if (absolute !== root && !absolute.startsWith(root + path.sep)) {
      continue;
    }
  }
}
```

```

    if (!isAllowedFile(absolute)) {
        continue;
    }
    let stats;
    try {
        stats = fs.statSync(absolute);
    } catch (error) {
        continue;
    }
    if (!stats.isFile()) {
        continue;
    }
    const name = path.relative(root, absolute).split(path.sep).join('/');
    try {
        upsertSessionFile(threadId, deviceId, absolute, name, stats.size, stats.mtime.toISOString());
    } catch (error) {
        console.error(`[gateway] 登记会话文件失败: ${error instanceof Error ? error.message : error}`);
    }
}
}

// 工作区扫描时跳过的目录（避免遍历依赖/构建产物）。
const SKIPPED_DIRS = new Set([
    'node_modules', '.git', '.svn', '.hg', 'build', '.build',
    'cache', 'dist', 'out', 'target', 'cert', 'keys', 'data'
]);

function walkAllowedFiles(dir, depth, onFile) {
    if (depth < 0) {
        return;
    }
    let entries;
    try {
        entries = fs.readdirSync(dir, { withFileTypes: true });
    } catch (error) {
        return;
    }
    for (const entry of entries) {
        const full = path.join(dir, entry.name);
        if (entry.isDirectory()) {
            if (entry.name.startsWith('.') || SKIPPED_DIRS.has(entry.name)) {
                continue;
            }
            walkAllowedFiles(full, depth - 1, onFile);
        } else if (entry.isFile() && !entry.name.startsWith('.') && isAllowedFile(entry.name)) {
            onFile(full);
        }
    }
}

```

```

/**
 * 汇总某设备可见的文件：已登记（session_files）的文件 ∪ 本设备专属工作区目录下的实际文件。
 * 工作区目录按 sha256(deviceId) 生成，天然按设备隔离；两者都限定在设备范围内。
 * 对外统一用扁平文件名（basename），按 basename 去重。
 */
function collectDeviceFiles(deviceId) {
  const filesRoot = resolveFilesRoot();
  if (!filesRoot) {
    return [];
  }
  const root = path.resolve(filesRoot);
  const byName = new Map();

  const add = (absolute) => {
    if (absolute !== root && !absolute.startsWith(root + path.sep)) {
      return;
    }
    if (!isAllowedFile(absolute)) {
      return;
    }
    let stats;
    try {
      stats = fs.statSync(absolute);
    } catch (error) {
      return;
    }
    if (!stats.isFile()) {
      return;
    }
    const name = path.basename(absolute);
    if (name.length === 0 || byName.has(name)) {
      return;
    }
    byName.set(name, { name, path: absolute, size: stats.size, modifiedAt: stats.mtime.toISOString() });
  };

  // 1) 已登记文件
  for (const row of listSessionFileRecords(deviceId)) {
    add(path.resolve(root, row.name));
  }

  // 2) 本设备专属工作区目录下的实际文件
  const workspace = deviceWorkspace(root, deviceId);
  if (workspace) {
    walkAllowedFiles(workspace, 4, add);
  }

  return Array.from(byName.values()).sort((left, right) => left.name.localeCompare(right.name));
}

```

```
function createGatewayHandler(options = {}) {
  const tasks = new Map();
  const opencodeClient = options.opencodeClient ?? new OpenCodeServerClient();

  setInterval(() => {
    cleanupExpiredNonces();
  }, 60 * 1000).unref();

  return async (request, response) => {
    try {
      const url = new URL(request.url ?? '/', 'http://localhost');

      if (request.method === 'OPTIONS') {
        jsonResponse(response, 204, {});
        return;
      }

      // —— Health ——
      if (request.method === 'GET' && url.pathname === '/health') {
        jsonResponse(response, 200, { status: 'ok', sessionName: 'opencode-main' });
        return;
      }

      // —— Auth: Register (get activation challenge) ——
      if (request.method === 'POST' && url.pathname === '/api/auth/register') {
        const ip = getClientIp(request);
        if (!registerLimiter.check(ip)) {
          errorResponse(response, 429, 'rate_limited', 'Too many registration attempts');
          return;
        }

        const body = await readJson(request);
        const deviceId = typeof body.deviceId === 'string' ? body.deviceId.trim() : '';
        const publicKey = typeof body.publicKey === 'string' ? body.publicKey.trim() : '';

        if (!deviceId || !publicKey) {
          errorResponse(response, 400, 'invalid_request', 'deviceId and publicKey are required');
          return;
        }

        // 开放注册：首次注册时服务器为该设备随机分配注册码并绑定（此后不可更改/换绑）。
        const registrationCode = ensureDeviceRegistrationCode(deviceId);
        const challenge = createActivationChallenge(deviceId, publicKey);
        jsonResponse(response, 200, { ...challenge, registrationCode });
        return;
      }

      // —— Auth: Activate (complete registration with signature) ——
      if (request.method === 'POST' && url.pathname === '/api/auth/activate') {
        const body = await readJson(request);
      }
    }
  };
}
```

```
const deviceId = typeof body.deviceId === 'string' ? body.deviceId.trim() : '';
const activationToken = typeof body.activationToken === 'string' ? body.activationToken.trim() : '';
const signedChallenge = typeof body.signedChallenge === 'string' ? body.signedChallenge.trim() : '';

if (!deviceId || !activationToken || !signedChallenge) {
  errorResponse(response, 400, 'invalid_request', 'deviceId, activationToken, and signedChallenge are
required');
  return;
}

if (!activateLimiter.check(activationToken)) {
  errorResponse(response, 429, 'rate_limited', 'Too many activation attempts');
  return;
}

const result = await verifyActivation(deviceId, activationToken, signedChallenge);
if (result.error) {
  const statusMap = {
    invalid_token: 400, device_mismatch: 403, token_used: 400,
    token_expired: 400, no_public_key: 500, invalid_signature: 403
  };
  errorResponse(response, statusMap[result.error] ?? 400, result.error, result.error);
  return;
}

jsonResponse(response, 200, result);
return;
}

// —— Auth: Challenge (get login nonce) ——
if (request.method === 'POST' && url.pathname === '/api/auth/challenge') {
  const body = await readJson(request);
  const deviceId = typeof body.deviceId === 'string' ? body.deviceId.trim() : '';

  if (!deviceId) {
    errorResponse(response, 400, 'invalid_request', 'deviceId is required');
    return;
  }

  if (!challengeLimiter.check(deviceId)) {
    errorResponse(response, 429, 'rate_limited', 'Too many challenge requests');
    return;
  }

  const result = createLoginChallenge(deviceId);
  if (!result) {
    errorResponse(response, 401, 'device_not_found', 'Device not registered or revoked');
    return;
  }

  jsonResponse(response, 200, result);
}
```

```
    return;
  }

  // —— Auth: Verify (login with signature) ——
  if (request.method === 'POST' && url.pathname === '/api/auth/verify') {
    const body = await readJson(request);
    const deviceId = typeof body.deviceId === 'string' ? body.deviceId.trim() : '';
    const nonce = typeof body.nonce === 'string' ? body.nonce.trim() : '';
    const signature = typeof body.signature === 'string' ? body.signature.trim() : '';

    if (!deviceId || !nonce || !signature) {
      errorResponse(response, 400, 'invalid_request', 'deviceId, nonce, and signature are required');
      return;
    }

    if (!verifyLimiter.check(deviceId)) {
      errorResponse(response, 429, 'rate_limited', 'Too many verification attempts');
      return;
    }

    const result = await verifyLogin(deviceId, nonce, signature);
    if (result.error) {
      const statusMap = {
        invalid_nonce: 400, device_mismatch: 403, nonce_used: 400,
        nonce_expired: 400, device_not_found: 401, invalid_signature: 403
      };
      errorResponse(response, statusMap[result.error] ?? 400, result.error, result.error);
      return;
    }

    jsonResponse(response, 200, result);
    return;
  }

  // —— Auth: Refresh ——
  if (request.method === 'POST' && url.pathname === '/api/auth/refresh') {
    const body = await readJson(request);
    const refreshToken = typeof body.refreshToken === 'string' ? body.refreshToken.trim() : '';

    if (!refreshToken) {
      errorResponse(response, 400, 'invalid_request', 'refreshToken is required');
      return;
    }

    if (!refreshLimiter.check(refreshToken)) {
      errorResponse(response, 429, 'rate_limited', 'Too many refresh attempts');
      return;
    }

    const result = refreshAccessToken(refreshToken);
```

```
    if (result.error) {
      errorResponse(response, 401, result.error, result.error);
      return;
    }

    jsonResponse(response, 200, result);
    return;
  }

  // —— Auth: Logout ——
  if (request.method === 'POST' && url.pathname === '/api/auth/logout') {
    const body = await readJson(request);
    const refreshToken = typeof body.refreshToken === 'string' ? body.refreshToken.trim() : '';
    if (refreshToken) {
      logoutDevice(refreshToken);
    }
    jsonResponse(response, 200, { ok: true });
    return;
  }

  // —— Auth: Reregister ——
  if (request.method === 'POST' && url.pathname === '/api/auth/reregister') {
    const ip = getClientIp(request);
    if (!reregisterLimiter.check(ip)) {
      errorResponse(response, 429, 'rate_limited', 'Too many reregistration attempts');
      return;
    }

    const body = await readJson(request);
    const deviceId = typeof body.deviceId === 'string' ? body.deviceId.trim() : '';
    const publicKey = typeof body.publicKey === 'string' ? body.publicKey.trim() : '';
    const deviceName = typeof body.deviceName === 'string' ? body.deviceName.trim() : '';
    const mode = typeof body.mode === 'string' ? body.mode.trim() : 'reset';

    if (!deviceId || !publicKey) {
      errorResponse(response, 400, 'invalid_request', 'deviceId and publicKey are required');
      return;
    }

    // 注册码无需输入: 沿用该设备首次注册时自动分配的码。
    const result = await reregisterDevice(deviceId, publicKey, deviceName, mode);
    if (result.error) {
      errorResponse(response, 401, result.error, result.error);
      return;
    }

    jsonResponse(response, 200, result);
    return;
  }
}
```

```

// —— Auth: Me ——
if (request.method === 'GET' && url.pathname === '/api/auth/me') {
  const auth = requireAuth(request, response);
  if (!auth) return;
  jsonResponse(response, 200, {
    deviceId: auth.deviceId,
    sessionId: 'opencode-main',
    registrationCode: ensureDeviceRegistrationCode(auth.deviceId)
  });
  return;
}

// —— ICE Config ——
if (request.method === 'GET' && url.pathname === '/api/turn/ice-config') {
  const auth = requireAuth(request, response);
  if (!auth) return;

  const turnHost = process.env.COTURN_HOST ?? '127.0.0.1';
  const turnPort = process.env.COTURN_PORT ?? '3478';
  const turnUser = process.env.COTURN_USER ?? 'codexrouter';
  const turnPass = process.env.COTURN_PASS ?? '';

  const iceServers = [{ url: `stun:${turnHost}:${turnPort}` }];
  if (turnPass.length > 0) {
    iceServers.push({ url: `turn:${turnHost}:${turnPort}`, username: turnUser, credential: turnPass });
  }
  jsonResponse(response, 200, { iceServers });
  return;
}

// —— Projects ——
if (request.method === 'GET' && url.pathname === '/api/projects') {
  const auth = requireAuth(request, response);
  if (!auth) return;
  jsonResponse(response, 200, { items: [PROJECT] });
  return;
}

// —— OpenCode Resumes ——
if (request.method === 'GET' && url.pathname === '/api/opencode/resumes') {
  const auth = requireAuth(request, response);
  if (!auth) return;

  const limit = Number(url.searchParams.get('limit') ?? '20');
  const allItems = await opencodeClient.listResumes({ limit: 1000 });
  // 归属持久化在 SQLite, gateway 重启后依然只列表本设备会话。
  const deviceThreadIds = new Set(listDeviceSessions(auth.deviceId));
  const items = allItems
    .filter((item) => deviceThreadIds.has(item.id))
    .slice(0, Number.isFinite(limit) && limit > 0 ? limit : 20);

```

```

    jsonResponse(response, 200, { items });
    return;
}

if (request.method === 'POST' && url.pathname === '/api/opencode/resumes') {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const filesRoot = resolveFilesRoot();
    if (!filesRoot) {
        errorResponse(response, 500, 'files_root_unset', 'FILES_ROOT or OPENCODE_WORKDIR is not configured');
        return;
    }

    // 设备专属工作目录: 强制会话 cwd 指向该设备目录, 实现文件天然按设备隔离。
    const workspace = deviceWorkspace(filesRoot, auth.deviceId);
    if (!workspace) {
        errorResponse(response, 400, 'invalid_device', 'Device id is required');
        return;
    }
    try {
        fs.mkdirSync(workspace, { recursive: true });
    } catch (error) {
        errorResponse(response, 500, 'workspace_create_failed', 'Could not create device workspace directory');
        return;
    }

    const cwd = workspace;
    // directory 让 opencode 把该会话的读写限制在设备工作区 (文件隔离的关键)。
    const session = await opencodeClient.startResume({ cwd, directory: workspace });
    if (auth.deviceId && session.threadId) {
        saveDeviceSession(session.threadId, auth.deviceId);
    }
    jsonResponse(response, 200, session);
    return;
}

const opencodeResumeMatch = url.pathname.match(/^\/api\/opencode\/resumes\/([^\s]+)\/resume$/);
if (request.method === 'POST' && opencodeResumeMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const threadId = decodeURIComponent(opencodeResumeMatch[1]);
    if (threadId.trim().length === 0) {
        errorResponse(response, 400, 'invalid_request', 'Thread id is required');
        return;
    }
    if (!requireThreadOwnership(response, auth.deviceId, threadId)) return;
    const directory = deviceWorkspace(resolveFilesRoot(), auth.deviceId) ?? '';
    const session = await opencodeClient.readResume({ threadId, directory });
}

```

```
    recordSessionFiles(auth.deviceId, threadId, session.files);
    jsonResponse(response, 200, session);
    return;
}

const opencodeArchiveResumeMatch = url.pathname.match(/^\/api\/opencode\/resumes\/([^\/]+)\archive$/);
if (request.method === 'POST' && opencodeArchiveResumeMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const threadId = decodeURIComponent(opencodeArchiveResumeMatch[1]);
    if (threadId.trim().length === 0) {
        errorResponse(response, 400, 'invalid_request', 'Thread id is required');
        return;
    }
    if (!requireThreadOwnership(response, auth.deviceId, threadId)) return;
    await opencodeClient.archiveResume({ threadId });
    deleteDeviceSession(threadId);
    jsonResponse(response, 200, { ok: true });
    return;
}

const opencodeRenameResumeMatch = url.pathname.match(/^\/api\/opencode\/resumes\/([^\/]+)\name$/);
if (request.method === 'POST' && opencodeRenameResumeMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const threadId = decodeURIComponent(opencodeRenameResumeMatch[1]);
    const body = await readJson(request);
    const name = typeof body.name === 'string' ? body.name.trim() : '';
    if (threadId.trim().length === 0 || name.length === 0) {
        errorResponse(response, 400, 'invalid_request', 'Thread id and name are required');
        return;
    }
    if (!requireThreadOwnership(response, auth.deviceId, threadId)) return;
    const directory = deviceWorkspace(resolveFilesRoot(), auth.deviceId) ?? '';
    await opencodeClient.renameResume({ threadId, name, directory });
    jsonResponse(response, 200, { ok: true });
    return;
}

const opencodeDeleteResumeMatch = url.pathname.match(/^\/api\/opencode\/resumes\/([^\/]*)$/);
if (request.method === 'DELETE' && opencodeDeleteResumeMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const threadId = decodeURIComponent(opencodeDeleteResumeMatch[1]);
    if (threadId.trim().length === 0) {
        errorResponse(response, 400, 'invalid_request', 'Thread id is required');
        return;
    }
}
```

```

    }
    if (!requireThreadOwnership(response, auth.deviceId, threadId)) return;
    await opencodeClient.deleteResume({ threadId });
    deleteDeviceSession(threadId);
    jsonResponse(response, 200, { ok: true });
    return;
}

const opencodeMessageMatch = url.pathname.match(/^\/api\/opencode\/resumes\/([^\/]+)\/messages$/);
if (request.method === 'POST' && opencodeMessageMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const threadId = decodeURIComponent(opencodeMessageMatch[1]);
    const body = await readJson(request);
    const message = typeof body.message === 'string' ? body.message : '';
    if (threadId.trim().length === 0 || message.trim().length === 0) {
        errorResponse(response, 400, 'invalid_request', 'Thread id and message are required');
        return;
    }
    if (!requireThreadOwnership(response, auth.deviceId, threadId)) return;
    const directory = deviceWorkspace(resolveFilesRoot(), auth.deviceId) ?? '';
    const session = await opencodeClient.sendResumeMessage({ threadId, message, directory });
    recordSessionFiles(auth.deviceId, threadId, session.files);
    jsonResponse(response, 200, session);
    return;
}

// —— Tasks ——
const createTaskMatch = url.pathname.match(/^\/api\/projects\/([^\/]+)\/tasks$/);
if (request.method === 'POST' && createTaskMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const projectId = createTaskMatch[1];
    if (projectId !== PROJECT.id) {
        errorResponse(response, 404, 'project_not_found', 'Project not found');
        return;
    }

    const body = await readJson(request);
    const taskId = `task_${randomUUID()}`;
    const task = {
        id: taskId, taskId, projectId, status: 'queued',
        message: typeof body.message === 'string' ? body.message : '',
        sandbox: typeof body.sandbox === 'string' ? body.sandbox : 'workspace-write',
        networkEnabled: Boolean(body.networkEnabled),
        createdAt: new Date().toISOString()
    };
    tasks.set(taskId, task);
}

```

```

    jsonResponse(response, 200, { taskId, status: task.status });
    return;
}

const taskMatch = url.pathname.match(/^\/api\/tasks\/([^/]+)$/);
if (request.method === 'GET' && taskMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;

    const task = tasks.get(taskMatch[1]);
    if (!task) {
        errorResponse(response, 404, 'task_not_found', 'Task not found');
        return;
    }
    jsonResponse(response, 200, task);
    return;
}

// —— Files: List (本设备可见文件 = 登记文件 ∪ 设备专属工作区文件) ——
if (request.method === 'GET' && url.pathname === '/api/files') {
    const auth = requireAuth(request, response);
    if (!auth) return;
    if (!resolveFilesRoot()) {
        errorResponse(response, 500, 'files_root_unset', 'FILES_ROOT or OPENCODE_WORKDIR is not configured');
        return;
    }
    const items = collectDeviceFiles(auth.deviceId)
        .map((file) => ({ name: file.name, size: file.size, modifiedAt: file.modifiedAt }));
    jsonResponse(response, 200, { items });
    return;
}

// —— Files: Download (仅限本设备可见文件: 登记记录或设备专属工作区) ——
const downloadMatch = url.pathname.match(/^\/api\/files\/(.+)\.download$/);
if (request.method === 'GET' && downloadMatch) {
    const auth = requireAuth(request, response);
    if (!auth) return;
    if (!resolveFilesRoot()) {
        errorResponse(response, 500, 'files_root_unset', 'FILES_ROOT or OPENCODE_WORKDIR is not configured');
        return;
    }
}

let requestedName;
try {
    requestedName = decodeURIComponent(downloadMatch[1]);
} catch (error) {
    requestedName = downloadMatch[1];
}

// 在“本设备可见文件”里按扁平文件名(basename)匹配, 限定设备范围, 杜绝越权/穿越。

```

```

    const wanted = path.basename(requestedName);
    const match = collectDeviceFiles(auth.deviceId).find((file) => file.name === wanted);
    if (!match) {
        errorResponse(response, 404, 'file_not_found', 'File does not exist');
        return;
    }
    const target = match.path;
    if (!isAllowedFile(target)) {
        errorResponse(response, 400, 'invalid_file', 'File name is invalid or not allowed');
        return;
    }

    let stats;
    try {
        stats = fs.statSync(target);
    } catch (error) {
        errorResponse(response, 404, 'file_not_found', 'File does not exist');
        return;
    }
    if (!stats.isFile()) {
        errorResponse(response, 404, 'file_not_found', 'File does not exist');
        return;
    }

    // 流式发送，避免大文件整块读入内存。
    response.writeHead(200, {
        'content-type': mimeTypeForFile(target),
        'content-length': stats.size,
        'content-disposition': `attachment; filename="${encodeURIComponent(path.basename(target))}"`
    });
    const stream = fs.createReadStream(target);
    stream.on('error', () => {
        response.destroy();
    });
    response.on('close', () => {
        stream.destroy();
    });
    stream.pipe(response);
    return;
}

errorResponse(response, 404, 'not_found', 'Not found');
} catch (error) {
    console.error(`[gateway] ${request.method} ${request.url} 500:`, error);
    errorResponse(response, 500, 'gateway_unavailable', error instanceof Error ? error.message : 'Gateway error');
}
}
}

export function createGatewayListener(options = {}) {

```

```

const handler = createGatewayHandler(options);
if (options.tlsKey && options.tlsCert) {
  const createSecureServer = options.createSecureServer ?? https.createServer;
  return createSecureServer({ key: options.tlsKey, cert: options.tlsCert }, handler);
}
return http.createServer(handler);
}

export function createGatewayServer(options = {}) {
  return createGatewayListener(options);
}

export function startGatewayServer(options = {}) {
  const host = options.host ?? process.env.GATEWAY_HOST ?? DEFAULT_HOST;
  const port = Number(options.port ?? process.env.GATEWAY_PORT ?? DEFAULT_PORT);
  const httpPort = Number(options.httpPort ?? process.env.GATEWAY_HTTP_PORT ?? 8080);
  const tlsKeyPath = options.tlsKeyPath ?? process.env.GATEWAY_TLS_KEY;
  const tlsCertPath = options.tlsCertPath ?? process.env.GATEWAY_TLS_CERT;
  const tlsOptions = tlsKeyPath && tlsCertPath ? {
    tlsKey: fs.readFileSync(tlsKeyPath),
    tlsCert: fs.readFileSync(tlsCertPath)
  } : {};
  const handler = createGatewayHandler(options);

  const servers = [];

  if (tlsKeyPath && tlsCertPath) {
    const httpsServer = https.createServer({ key: tlsOptions.tlsKey, cert: tlsOptions.tlsCert }, handler);
    httpsServer.listen(port, host, () => {
      const address = httpsServer.address();
      const actualPort = typeof address === 'object' && address ? address.port : port;
      console.log(`AIRouter Gateway listening on https://${host}:${actualPort}`);
    });
    servers.push(httpsServer);
  }

  const httpServer = http.createServer(handler);
  httpServer.listen(httpPort, host, () => {
    const address = httpServer.address();
    const actualPort = typeof address === 'object' && address ? address.port : httpPort;
    console.log(`AIRouter Gateway listening on http://${host}:${actualPort}`);
  });
  servers.push(httpServer);

  return servers;
}

if (import.meta.url === `file://${process.argv[1]}`) {
  startGatewayServer();
}

```